ソフトウェア工学特論 講義資料

第4回 git入門：基本操作

- バージョン管理の概念とリポジトリ
- init / add / commit の基本操作
- ワーキングツリー / ステージ / リポジトリの3領域

目次

- バージョン管理とgitの概念
- 3つの領域と基本操作
- コミット履歴の確認

バージョン管理とgitの概念

関連ドキュメント: この節の解説

今回の目的と到達目標

今回の目的

- なぜバージョン管理が必要かを理解する
- リポジトリの概念を把握する
- init/add/commitを自分で実行できる

到達目標

- [R2-未到達] gitの基本操作（commit等）を実践できる

バージョン管理が必要な理由

- 「昨日の状態に戻したい」を仕組みで解決する
- ファイル名に `_v2` `_最終版` `_最終版_本当` を付けるのをやめる
- **誰が・いつ・何を・なぜ** 変更したかを記録に残す
- 複数人の変更を統合する基盤になる（第6週以降）
- 本講座では **すべての提出物をgitで管理する**

リポジトリ = プロジェクトの履歴貯蔵庫

- リポジトリ（repository）：変更履歴を保存する箱
- ローカルリポジトリ：自分のPC上
- リモートリポジトリ：GitHub等のサーバー上（第6週）
- `.git` ディレクトリに履歴がすべて保存される
- プロジェクトのルートで `git init` すると作成される

3つの領域と基本操作

関連ドキュメント: この節の解説

3つの領域

領域	役割	関連コマンド
ワーキングツリー	実際のファイル	編集そのもの
ステージ（index）	次のコミット候補	`git add`
リポジトリ	確定した履歴	`git commit`

変更は **ワーキングツリー** → **ステージ** → **リポジトリ** の順で進む。

基本コマンドの流れ

```
# プロジェクト初期化
git init

# 変更をステージに追加
git add README.md

# コミット（履歴に確定）
git commit -m "初回コミット: README作成"
```

`add` と `commit` を分けることで、**1コミットに含める変更を選べる。**

領域間の遷移

ファイル編集 → ステージへ追加 → リポジトリへ確定

1. **編集**: `README.md` を修正（ワーキングツリー）
2. **ステージ**: `git add README.md` （index に配置）
3. **コミット**: `git commit -m "..."` （履歴に確定）

確定後は `git log` で履歴を辿れる。取り消しも可能。

コミット履歴の確認

関連ドキュメント: この節の解説

状態と履歴を見るコマンド

コマンド	役割
`git status`	今どの領域に何があるか
`git diff`	ワーキングツリーの未ステージ差分
`git diff --staged`	ステージの差分
`git log`	コミット履歴
`git log --oneline`	履歴を1行で表示

迷ったら `git status` が鉄則。

良いコミットメッセージ

- **要点を先頭に**（1行目50字以内、何をしたか）
- 2行目は空行、3行目以降で理由や背景を書く
- 動詞で始める（「追加」「修正」「更新」）
- 1コミット = 1つの論理的変更
- レポート類も「章を追加」「誤字を修正」のように分けて記録する

レポート1の導入セクションを追加

チェーン店選定の理由を読み手に予告する段落として追加。

今回のまとめ

- バージョン管理は **誰が・いつ・何を・なぜ** の記録
- 3領域：ワーキングツリー / ステージ / リポジトリ
- 基本操作：init → add → commit
- 迷ったら `git status`

今回カバーしたMCC項目

- V-D-1 プログラミング
- V-D-4 コンピュータシステム

次回予告

- 第5回: gitの実践 — ブランチ・マージ・リベース